HACKATÓN · GRUPO 3

Perú Wallet

Cómo la construimos con agentes, skills y MCP

y cómo resolvimos la wallet contra el mock del grupo 3

OpenSpec · CrewAI (5 roles) · MCP local · guardrails · aprobación humana

23 de octubre de 2026

Cómo se ejecuta
python main.py -> http://127.0.0.1:8000

PUNTO DE PARTIDA

El reto y nuestra estrategia

El reto

Una wallet tipo Yape: registro, pago QR, reintentos seguros, fallas, soporte y conciliación sobre un mock con 32 operaciones. CrewAI y MCP son obligatorios.

Tres decisiones

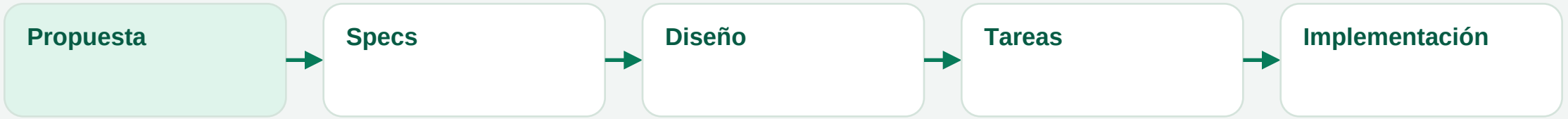
1. Especificar antes de construir (OpenSpec).
2. Una persona aprueba; los agentes proponen.
3. Nada es 'aprobado' sin evidencia observada.

Regla de oro

El mock del grupo 3 es la única autoridad financiera: ni los agentes ni la web inventan saldos, pagos o ledger. Lo que el mock no ofrece se declara como brecha, no se simula.

CÓMO PLANIFICAMOS

Spec-driven con OpenSpec



- 6 capacidades y 48 requisitos verificables, cada uno trazado a las historias W1-W6.
- 25/27 tareas completadas; cada una con su evidencia de verificación.
- Los recortes de demo se marcan DEMO, POSTERIOR o BLOQUEADO; las historias originales no se editan.
- El alcance fue aprobado por una persona antes de implementar (tarea 1.1, en la conversación); el registro firmado para el runner sigue pendiente. Jira solo con autorización aparte.

CINCO ROLES, UN ORDEN, ENTREGABLES DISTINTOS

El equipo de agentes CrewAI

1. Product Owner
Propuesta de alcance y trazabilidad W1-W6

→

2. Arquitecto/a
Matriz de contratos, estados y MCP

→

3. AI Engineer
Cambios por workspace y verificación

→

4. QA & Security
Matriz por criterio y dictamen

→

5. Presenter / Timekeeper
Guion, evidencia y limitaciones

- Secuencial: cada rol recibe las fuentes aprobadas y la entrega del anterior.
- Si un rol responde BLOQUEADO, los siguientes no corren: no se rellena con invenciones.
- Cada entregable tiene formato exacto y distingue hecho, propuesta y evidencia ejecutada.
- No usamos LangGraph: el flujo del hackatón es CrewAI; el laboratorio anterior quedó aparte (lab_main.py).

UNA SKILL POR AGENTE, CARGADA DE VERDAD

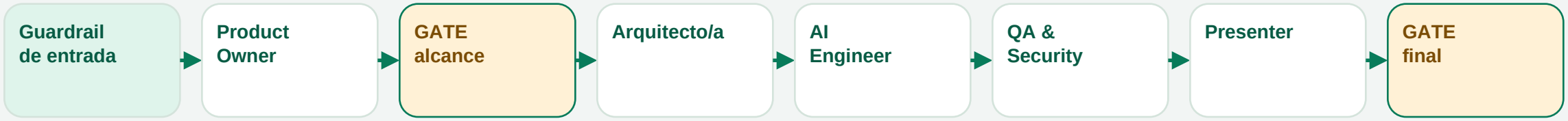
Skills especializadas por rol

Rol	Fuentes propias	Entregable	Límite clave
Product Owner	Historias W1-W6, epic, criterios	Alcance + trazabilidad + solicitud de aprobación	No aprueba su propia propuesta
Arquitecto/a	OpenAPI del mock, mapa de guardrails	Matriz de contratos y modelo de estados	El diseño no crea saldo ni ledger
AI Engineer	Alcance aprobado y matriz	Cambios por workspace + verificación	Sin documento_deuda_id ni reversos
QA & Security	Criterios y evidencia observada	Matriz con 4 etiquetas + dictamen	Sin evidencia no hay APROBADO
Presenter	Evidencias, brechas, guion	Guion, tiempos y limitaciones	PENDING no es éxito

Mecanismo: agents/common.py carga la skill en la backstory del agente. Si falta, es ilegible o está vacía se lanza SkillError: no hay instrucciones genéricas de respaldo.

RUNNER SECUENCIAL CON DOS GATES HUMANOS

Cómo corre el equipo



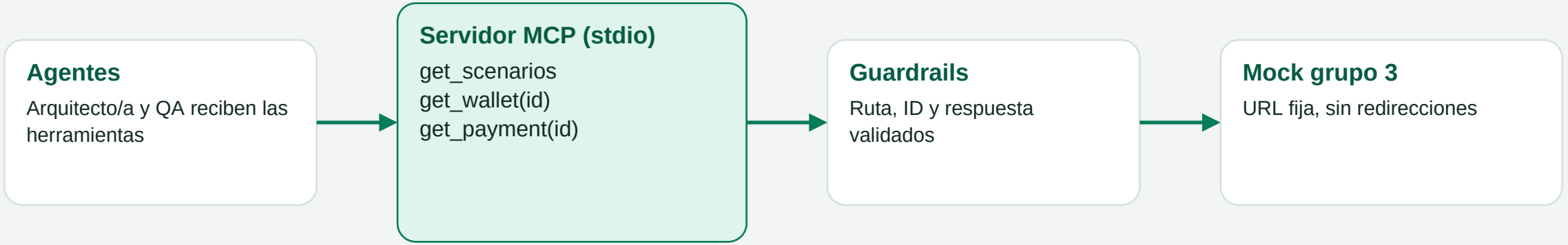
- El gate de alcance exige un registro firmado por una persona, ligado a la versión del alcance (hash de criterios, propuesta y specs); si algo cambia, hay que reaprobar.
- El gate final liga la decisión del Product Owner humano al digest de las entregas revisadas.
- Ningún agente puede aprobar: el runner solo lee los registros, nunca los crea.
- Dry-run: agentes reales con salidas simuladas; jamás devuelve 'ready'.

Estado real

Sin aprobación humana el runner se detiene en BLOQUEADO (código 2): es el comportamiento correcto, no un error.

CÓMO LOS AGENTES CONSULTAN EL MOCK

MCP local de solo lectura



Intento del agente	Código	Resultado
Reset del grupo o reversos	DEMO-TOOL-02	Rechazado, nunca llega al mock
Escritura / pago por MCP	DEMO-TOOL-03	Rechazado: requiere confirmación humana
Herramienta o URL no listada	DEMO-TOOL-01	Denegado por defecto
Respuesta con secreto o tarjeta	DEMO-OUT-01	Bloqueada; se informa como fallo

Evidencia: llamada real GET /scenarios por stdio (HTTP 200) y un agente CrewAI, con un LLM guionado, que consumió la respuesta real del MCP (no se probó con un modelo real). Un fallo del MCP se reporta como fallo, nunca como éxito.

UN GUARDRAIL VALIDA, LIMITA O BLOQUEA Y DEJA UN CÓDIGO

Guardrails: reglas, no opiniones

Entrada

DEMO-IN-01..05
Solo TEST-, campos permitidos, sin secretos ni tarjetas, IDs seguros y clave de 6 dígitos con hash y sal

Rutas y herramientas

DEMO-TOOL-01..03
19 permitidas · 7 con confirmación humana · 6 prohibidas. Lo no mapeado se deniega.

Salida

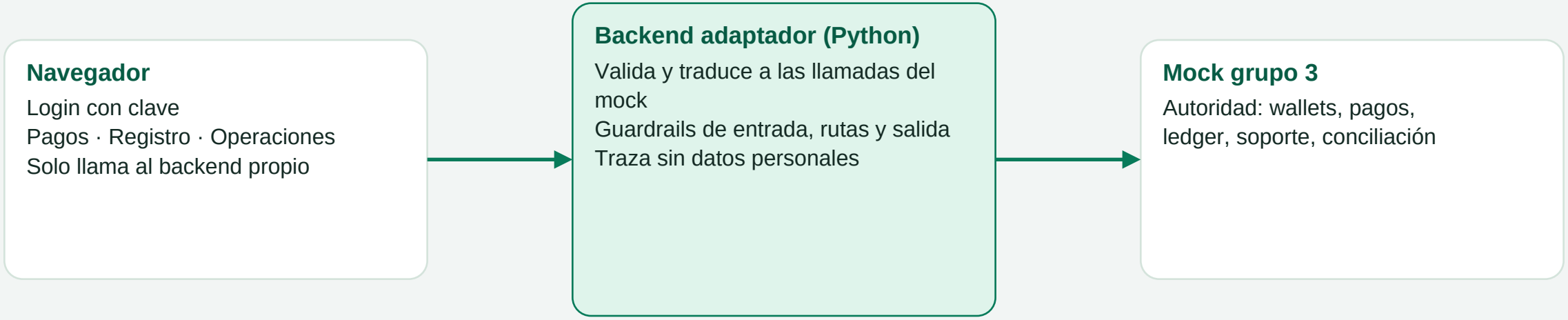
DEMO-OUT-01
Ninguna respuesta con secretos o tarjetas llega al navegador ni al agente.

Un solo mapa para todo

El mismo FEATURE_MAP de 32 operaciones gobierna el backend de la web, la lista permitida del MCP y la tabla visible en Operaciones. Una prueba falla si algún endpoint del contrato queda sin clasificar.

NAVEGADOR -> BACKEND ADAPTADOR -> MOCK

Cómo resolvimos la wallet



- Sin saldo ni estado de pago locales: el 'éxito' se muestra solo tras reconsultar al mock.
- Idempotency-Key por intento: reintento seguro, conflicto 409 y evento duplicado sin doble pago.
- Operaciones sensibles (pago, decisión KYC, alterar reporte) exigen confirmación explícita.
- Brecha honesta: el mock no recibe documento_deuda_id, así que el pago de una deuda concreta queda POSTERIOR.

LO QUE VE LA PERSONA

La wallet en uso

Registro / ingreso	Usuario TEST- con clave de 6 dígitos; si ya existe, ingresa; KYC simulado y decisión humana si hay revisión manual.
Pagos	Mi saldo, pagar a contacto o comercio varias veces, confirmación, 'Pago exitoso' solo tras consultar al mock, comprobante.
QR	Generar y resolver QR (con imagen), pagar con su qrId; vista de comercio con filtros.
Fallas	503, 504, QR vencido y rechazo con guía segura: reintentar solo cuando no se creó el pago.
Operaciones	Soporte de solo lectura, liquidaciones, ledger, conciliación e incidencias; alteración AMOUNT/CLEAR con confirmación.

PYTHON MAIN.PY

Los agentes también gobiernan el arranque

Etapa (orden de los agentes)	Qué verifica, sin llamar a un modelo	Resultado típico
1. Product Owner	Versión de alcance y aprobación humana inicial	ADVERTENCIA si falta
2. Arquitecto/a	32 operaciones clasificadas; reset y reversos prohibidos	OK
3. AI Engineer	Módulos compilan, web completa, 5 skills presentes	OK
4. QA & Security	Pruebas automatizadas de la demo	OK / FALLO
5. Presenter	Puerto libre, URL y guion	OK / FALLO

- Si ninguna etapa falla, levanta el servidor Python; con --strict exige la aprobación humana.
- --crew dry-run|real ejecuta además el runner CrewAI. Las etapas nunca inventan una aprobación.

HALLAZGOS QUE EL ANÁLISIS NO MOSTRABA

Lo que nos enseñó probar con el mock real

- El pago salía sin qrId: la resolución del QR no devuelve su ID. Lo derivamos del payload.
- El 503 real es PROVIDER_UNAVAILABLE (no PROVIDER_ERROR): la guía depende del estado HTTP.
- El 504 trae un status interno que no es el resultado: lo rotulamos como informado junto al error.
- El resumen de conciliación no separaba pagos con y sin diferencias (W6-04): lo completamos.
- El mock no tiene recarga: el saldo inicial es wallet-demo y se declara, no se simula.
- Un bug propio (clave de sessionStorage duplicada) rompió las llamadas: lo atrapó la prueba en navegador.

CIFRAS DE LA CORRIDA REAL

Resultados y brechas

25/27

tareas OpenSpec completas

48

requisitos verificables

129

pruebas automatizadas OK

40

criterios APROBADOS con evidencia

Brechas declaradas

6 criterios NO EJECUTADOS y 1 BLOQUEADO (pago de una deuda concreta, POSTERIOR).
CrewAI con modelo real no se ejecutó: Copilot sin autenticar; sí dry-run, MCP real y pruebas.
Sin reversos, devoluciones ni disputas.

Requiere a una persona

Aprobación de alcance (evidence/scope_approval.json).
Ensayo cronometrado de la demo.
Aprobación final del Product Owner.
El mock es compartido: otros grupos pueden dejar incidencias abiertas.

GRACIAS

Demo en vivo

- python main.py y abrir http://127.0.0.1:8000
- Registrarme con TEST-… y clave de 6 dígitos; luego Ingresar
- Pagar a un contacto y a un comercio y ver bajar el saldo
- Provocar un 503/504; mostrar el mapa de guardrails y el runner bloqueado sin aprobación

Respaldo

Si el mock no responde: se muestra el error tal cual, la evidencia de la corrida y los guardrails; no se simulan respuestas.